

intermediate states such as hash tables, aggregation buffers, and sorting structures are allocated from Velox's hierarchical *MemoryPool* system.

When memory usage approaches configured limits, the *MemoryArbitrator* coordinates memory reclamation. Spill-capable operators can write intermediate state to local storage, allowing large analytical queries to continue executing even when their working set exceeds available RAM.

This approach minimises out-of-memory failures while maintaining predictable query execution.

Result delivery and network shuffle:

After the final operator completes processing, the resulting columnar vectors are prepared for delivery.

For single-node execution, the vectors are returned directly to the calling engine. For distributed execution, the output may be serialised into engine-specific formats such as Presto Pages or Spark's *UnsafeRow* representation before being exchanged with downstream workers or returned to the parent engine.

From the perspective of Presto or Spark, Velox acts as a highly optimised execution backend that transforms an optimised physical plan into the final query results.

Where is Velox being used?

Velox is under active development but is already being integrated into more than a dozen different data systems at Meta and across the broader open source ecosystem. By providing a unified set of high-performance components, Velox is bridging the gaps between historically siloed workloads.

Here are some of the most prominent data management systems currently leveraging Velox.

Interactive analytics: Prestissimo (Presto C++): Presto is a distributed query engine that powers massive interactive SQL analytic workloads. Traditionally, Presto worker nodes ran on the Java Virtual Machine (JVM), which was historically prone

to operational issues and expensive garbage collection procedures. Through a project called Prestissimo, developers are replacing these traditional Java workers with a C++ process powered entirely by Velox. Prestissimo provides a drop-in replacement that takes a Presto plan fragment from a Java coordinator and hands it directly to Velox for native execution, completely bypassing the JVM on the worker nodes and delivering massive performance improvements.

Batch processing: Apache Spark (Spruce and Gluten): For heavy-duty, large scale batch processing and ETL (Extract, Transform, Load) workloads, Velox is deeply integrated into Apache Spark. This is achieved through two primary initiatives:

- *Spruce*: Meta's internal integration, which uses Spark's built-in script transform interface to offload query plan fragments to an external C++ process running Velox.
- *Apache Gluten*: An open source project originally created by Intel. Gluten acts as a bridge, creating a Java Native Interface (JNI) API based on the Apache Arrow data format and Substrait query plans. This decouples Spark's JVM from the execution engine, allowing Spark to seamlessly delegate compute-intensive tasks to Velox.

Artificial intelligence and machine learning: One of Velox's most ambitious capabilities is bridging the gap between traditional data analytics and artificial intelligence.

- *TorchArrow*: Velox serves as the backend execution engine for TorchArrow, a Python DataFrame library for data preprocessing in PyTorch. Because data preprocessing can consume up to 50% of the resources in an ML workload, using Velox standardises these operations and provides tremendous efficiency wins.
- *F3*: Meta's feature engineering framework, F3, is also unifying its

execution engine with Velox. This allows developers to seamlessly extract and generate features for ML algorithms across both offline (batch) and real-time environments.

Real-world performance: Velox in action

To prove that Velox C++ design translates to real-world speed, Velox has been subjected to rigorous benchmarks across multiple execution environments. The results demonstrate that by replacing traditional Java execution layers or leveraging specialised hardware, Velox delivers staggering performance gains and massive infrastructure cost savings.

The CPU benchmark: Presto

Java vs Velox C++ (Prestissimo): The first major test of Velox's efficiency was executed at Meta, comparing the traditional Java based Presto engine against Prestissimo (Presto workers powered by Velox's C++ engine).

- *Standard TPC-H (3TB Scale)*: In a warm-cache cluster of 80 nodes, CPU-bound queries saw near-order-of-magnitude improvements. Query 1 (Q1) achieved an 8.4x wall-time speedup (dropping from 42 seconds to just 5 seconds), while Query 6 (Q6) saw a 9x speedup (dropping from 9 seconds to 1 second). Shuffle-heavy queries like Q13 and Q19 also doubled in speed.
- *Shadowing real production traffic*: To move past synthetic benchmarks, Meta replayed real production traffic from various interactive analytical tools. The Velox-powered C++ engine delivered an average speedup of 6x to 7x, with a significant portion of queries running over 10x faster than the traditional Java engine.
- *The 3x server consolidation win*: Perhaps the most impactful result for infrastructure teams is cluster capacity. In shadowing tests, the Velox-based stack supported identical production workloads with equal or better performance using